

EX-13 Write the Python program for Minimax Algorithm for Gaming

AIM

To develop a Python program to implement the **Minimax Algorithm** for gaming, enabling the computer to choose the optimal move by maximizing its score and minimizing the opponent's score.

ALGORITHM

Step 1: Start the program.

Step 2: Define the game tree with terminal node values.

Step 3: Set the root node as the starting position.

Step 4: If the current node is a terminal node, return its value.

Step 5: If it is the maximizing player's turn, choose the maximum value among all child nodes.

Step 6: If it is the minimizing player's turn, choose the minimum value among all child nodes.

Step 7: Recursively evaluate all possible moves until the terminal nodes are reached.

Step 8: Return the optimal value to the root node.

Step 9: Display the optimal value selected by the Minimax algorithm.

Step 10: Stop the program.

PROGRAM CODE

```
import math
```

```
def minimax(depth, nodeIndex, maximizingPlayer, values, maxDepth):
```

```
    if depth == maxDepth:
```

```
return values[nodeIndex]
```

```
if maximizingPlayer:
```

```
    return max(  
        minimax(depth + 1, nodeIndex * 2, False, values, maxDepth),  
        minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)  
    )
```

```
else:
```

```
    return min(  
        minimax(depth + 1, nodeIndex * 2, True, values, maxDepth),  
        minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)  
    )
```

```
values = list(map(int, input("Enter 8 terminal values: ").split()))
```

```
height = int(math.log2(len(values)))
```

```
print("\nThe optimal value is:",  
      minimax(0, 0, True, values, height))
```

OUTPUT

```
=====
Enter 8 terminal values: 3 5 2 9 12 5 23 23

The optimal value is: 12
.
```

RESULT

The Python program successfully implements the **Minimax Algorithm** and determines the optimal move by recursively selecting the maximum and minimum values at alternate levels of the game tree.